ITA0613 - MACHINE LEARNING

Google Colab Coding - Questions (a) to (g)

Question (a)

```
# ITA0613 Machine Learning Assignment
# Large-Scale Instance-Based and Statistical Learning Pipeline for Climate-Resilient Crop Yield Forecasting
# Google Colab-ready. Core ML algorithms are implemented from first principles using NumPy/Pandas.

import sys, subprocess
subprocess.run([sys.executable, "-m", "pip", "install", "-q", "kagglehub"], check=False)

import os, glob, time, tracemalloc, math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# ------------------------------
# 1. DATASET: SAME DATA FOR ALL 7 QUESTIONS
# ------------------------------
DATASET = "sujaldhanwani/fao-crop-dataset"

try:
    import kagglehub
    root = kagglehub.dataset_download(DATASET)
    csvs = glob.glob(os.path.join(root, "**", "*.csv"), recursive=True)
except Exception as e:
    print("Automatic Kaggle download failed:", e)
    print("Upload the CSV manually below.")
    from google.colab import files
    uploaded = files.upload()
    csvs = [os.path.join("/content", f) for f in uploaded.keys() if f.lower().endswith(".csv")]

assert csvs, "No CSV file found."
print("CSV files found:", csvs)
csv_path = csvs[0]
df = pd.read_csv(csv_path)
print("Raw shape:", df.shape)
display(df.head())

# ------------------------------
# 2. COLUMN NORMALIZATION
# ------------------------------
def norm(s):
    return ''.join(ch for ch in str(s).lower() if ch.isalnum())

def find_col(columns, exact=(), contains=()):
    nmap = {c: norm(c) for c in columns}
    for key in exact:
        for c, n in nmap.items():
            if n == norm(key):
                return c
    for key in contains:
        for c, n in nmap.items():
            if norm(key) in n:
                return c
    return None

cols = list(df.columns)

year_col = find_col(cols, exact=("year","crop_year","cropyear"))
state_col = find_col(cols, exact=("state","state_name","statename"))
district_col = find_col(cols, exact=("district","district_name","districtname"))
crop_col = find_col(cols, exact=("crop","crop_name","cropname","item"))
season_col = find_col(cols, exact=("season","crop_season","season_name"))
rain_col = find_col(cols, contains=("rainfall","rain"))
temp_col = find_col(cols, contains=("temperature","temp"))
hum_col = find_col(cols, contains=("humidity","humid"))
yield_col = find_col(cols, exact=("yield","crop_yield","yield_kg_ha","yield_kg_ha_"), contains=("yield",
"productivity"))
n_col = find_col(cols, exact=("n","nitrogen","soil_n","nitrogen_kg_ha"), contains=("nitrogen","soiln"))
p_col = find_col(cols, exact=("p","phosphorus","soil_p","phosphorus_kg_ha"), contains=("phosphorus","soilp"))
k_col = find_col(cols, exact=("k","potassium","soil_k","potassium_kg_ha"), contains=("potassium","soilk"))
```

```
ph_col = find_col(cols, exact=("ph","soil_ph","ph_value"), contains=("soilph","ph"))

print({
    "year": year_col, "state": state_col, "district": district_col,
    "crop": crop_col, "season": season_col, "rainfall": rain_col,
    "temperature": temp_col, "humidity": hum_col, "yield": yield_col,
    "N": n_col, "P": p_col, "K": k_col, "pH": ph_col
})

required = [rain_col, temp_col, yield_col]
if any(c is None for c in required):
    raise ValueError("Could not identify rainfall, temperature, and yield columns. Print df.columns and
update the mapping.")

# Missing optional variables are created as NaN and imputed; this keeps the pipeline runnable.
for name, col in {
    "year":year_col, "state":state_col, "district":district_col,
    "crop":crop_col, "season":season_col, "rainfall":rain_col,
    "temperature":temp_col, "humidity":hum_col, "yield":yield_col,
    "N":n_col, "P":p_col, "K":k_col, "pH":ph_col
}.items():
    if col is None:
        df[name] = np.nan
    elif col != name:
        df[name] = df[col]

# ----------------------------
# 3. CLEANING + IMPUTATION + FEATURE ENGINEERING (Q a)
# ----------------------------
df = df.drop_duplicates().copy()

num_cols = ["year","rainfall","temperature","humidity","N","P","K","pH","yield"]
for c in num_cols:
    df[c] = pd.to_numeric(df[c], errors="coerce")

# Remove impossible target values and impute numeric weather/soil fields with group medians.
df.loc[df["yield"] <= 0, "yield"] = np.nan

group_cols = [c for c in ["district","crop"] if c in df.columns and df[c].notna().any()]
if group_cols:
    for c in ["rainfall","temperature","humidity","N","P","K","pH"]:
        med = df.groupby(group_cols)[c].transform("median")
        df[c] = df[c].fillna(med)
for c in ["rainfall","temperature","humidity","N","P","K","pH"]:
    df[c] = df[c].fillna(df[c].median())

df = df.dropna(subset=["yield","rainfall","temperature"]).reset_index(drop=True)

# At least three derived features:
# 1) Growing Degree Days (annual proxy because the public dataset is annual)
# 2) Rainfall anomaly index
# 3) Heat stress index
# 4) Soil fertility index
base_temp = 10.0
df["GDD_proxy"] = np.maximum(df["temperature"] - base_temp, 0) * 365.0

rain_group = [c for c in ["district"] if c in df.columns and df[c].notna().any()]
if rain_group:
    rain_mean = df.groupby(rain_group)["rainfall"].transform("mean")
else:
    rain_mean = df["rainfall"].mean()
df["rainfall_anomaly"] = (df["rainfall"] - rain_mean) / (rain_mean.abs() + 1e-9)

df["heat_stress"] = np.maximum(df["temperature"] - 30.0, 0.0)

soil_cols = ["N","P","K"]
df["soil_fertility_index"] = df[soil_cols].mean(axis=1)

# Replace categorical missing values.
for c in ["state","district","crop","season"]:
    df[c] = df[c].astype(str).fillna("Unknown")

print("Cleaned shape:", df.shape)
print(df[["GDD_proxy","rainfall_anomaly","heat_stress","soil_fertility_index"]].describe())
display(df.head())

# ----------------------------
# 4. NUMERIC FEATURE MATRIX
# ----------------------------
```

```
numeric_features = [
    "rainfall","temperature","humidity","N","P","K","pH",
    "GDD_proxy","rainfall_anomaly","heat_stress","soil_fertility_index"
]
X_num = df[numeric_features].to_numpy(dtype=float)
y = df["yield"].to_numpy(dtype=float)

# Manual standardization, not sklearn.
mu = X_num.mean(axis=0)
sigma = X_num.std(axis=0)
sigma[sigma == 0] = 1.0
X = (X_num - mu) / sigma

# ----------------------------
# 5. DISTRICT-HOLDOUT SPLIT
# ----------------------------
rng = np.random.default_rng(42)
districts = df["district"].dropna().unique()
rng.shuffle(districts)
n_test_dist = max(1, int(0.20 * len(districts)))
test_districts = set(districts[:n_test_dist])

is_test = df["district"].isin(test_districts).to_numpy()
remaining = np.where(~is_test)[0]
rng.shuffle(remaining)
n_val = max(1, int(0.20 * len(remaining)))
val_idx = remaining[:n_val]
train_idx = remaining[n_val:]

X_train, y_train = X[train_idx], y[train_idx]
X_val, y_val = X[val_idx], y[val_idx]
X_test, y_test = X[is_test], y[is_test]

print("Train/Val/Test:", X_train.shape, X_val.shape, X_test.shape)

# ----------------------------
# 6. FROM-SCRATCH DISTANCE METRICS
# ----------------------------
def euclidean_distances(A, b):
    return np.sqrt(np.sum((A - b)**2, axis=1))

cov = np.cov(X_train, rowvar=False)
cov += 1e-6 * np.eye(cov.shape[0])
cov_inv = np.linalg.pinv(cov)

def mahalanobis_distances(A, b, inv_cov):
    d = A - b
    return np.sqrt(np.einsum("ij,jk,ik->i", d, inv_cov, d))

# ----------------------------
# 7. FROM-SCRATCH KNN REGRESSOR (Q b)
# ----------------------------
def knn_predict(Xtr, ytr, Xq, k=5, metric="euclidean", inv_cov=None, chunk_size=256):
    preds = np.empty(len(Xq), dtype=float)
    for start in range(0, len(Xq), chunk_size):
        stop = min(start + chunk_size, len(Xq))
        Q = Xq[start:stop]
        out = []
        for q in Q:
            if metric == "euclidean":
                d = euclidean_distances(Xtr, q)
            else:
                d = mahalanobis_distances(Xtr, q, inv_cov)
            kk = min(k, len(d))
            idx = np.argpartition(d, kk-1)[:kk]
            # inverse-distance weighted mean; this is still a kNN regressor.
            w = 1.0 / (d[idx] + 1e-8)
            out.append(np.sum(w * ytr[idx]) / np.sum(w))
        preds[start:stop] = out
    return preds

def rmse(a,b): return float(np.sqrt(np.mean((a-b)**2)))
def mae(a,b): return float(np.mean(np.abs(a-b)))
def r2(a,b):
    return float(1 - np.sum((a-b)**2) / (np.sum((a-a.mean())**2) + 1e-12))

k_values = [1,3,5,7,9,15]
knn_curve = []
# Use a bounded validation subset for practical Colab runtime.
```

```
val_cap = min(1500, len(X_val))
vsub = np.arange(val_cap)
for k in k_values:
    pred = knn_predict(X_train, y_train, X_val[vsub], k=k, metric="euclidean")
    knn_curve.append(rmse(y_val[vsub], pred))
best_k = k_values[int(np.argmin(knn_curve))]
print("Validation RMSE by k:", dict(zip(k_values, knn_curve)))
print("Chosen k:", best_k)

# Compare Euclidean and Mahalanobis on the held-out district set.
test_cap = min(1500, len(X_test))
pred_e = knn_predict(X_train, y_train, X_test[:test_cap], k=best_k, metric="euclidean")
pred_m = knn_predict(X_train, y_train, X_test[:test_cap], k=best_k, metric="mahalanobis", inv_cov=cov_inv)

knn_results = pd.DataFrame({
    "Metric":["Euclidean","Mahalanobis"],
    "RMSE":[rmse(y_test[:test_cap],pred_e), rmse(y_test[:test_cap],pred_m)],
    "MAE":[mae(y_test[:test_cap],pred_e), mae(y_test[:test_cap],pred_m)],
    "R2":[r2(y_test[:test_cap],pred_e), r2(y_test[:test_cap],pred_m)]
})
display(knn_results)

# ------------------------------
# 8. LOCALLY WEIGHTED REGRESSION (Q c)
# ------------------------------
def lwr_predict(Xtr, ytr, Xq, bandwidth=1.0, ridge=1e-3, max_train=5000):
    # For runtime, cap the local reference set while keeping the final pipeline reproducible.
    if len(Xtr) > max_train:
        sel = np.linspace(0, len(Xtr)-1, max_train, dtype=int)
        Xt, yt = Xtr[sel], ytr[sel]
    else:
        Xt, yt = Xtr, ytr

    Xt1 = np.c_[np.ones(len(Xt)), Xt]
    preds = np.empty(len(Xq))
    I = np.eye(Xt1.shape[1])
    I[0,0] = 0.0
    for i, q in enumerate(Xq):
        d2 = np.sum((Xt-q)**2, axis=1)
        w = np.exp(-d2 / (2 * bandwidth**2))
        W = w[:,None]
        A = Xt1.T @ (W * Xt1) + ridge * I
        b = Xt1.T @ (w * yt)
        beta = np.linalg.solve(A, b)
        preds[i] = np.r_[1.0, q] @ beta
    return preds

bandwidths = [0.25,0.5,1.0,2.0,4.0]
lwr_curve = []
for bw in bandwidths:
    pred = lwr_predict(X_train, y_train, X_val[vsub], bandwidth=bw)
    lwr_curve.append(rmse(y_val[vsub], pred))
best_bw = bandwidths[int(np.argmin(lwr_curve))]
lwr_pred = lwr_predict(X_train, y_train, X_test[:test_cap], bandwidth=best_bw)

comparison = pd.DataFrame({
    "Model":[f"kNN (k={best_k})", f"LWR (h={best_bw})"],
    "RMSE":[rmse(y_test[:test_cap],pred_e), rmse(y_test[:test_cap],lwr_pred)],
    "MAE":[mae(y_test[:test_cap],pred_e), mae(y_test[:test_cap],lwr_pred)],
    "R2":[r2(y_test[:test_cap],pred_e), r2(y_test[:test_cap],lwr_pred)]
})
display(comparison)

# ------------------------------
# 9. CANDIDATE-ELIMINATION / VERSION SPACE (Q d)
# ------------------------------
ce = df.copy()
ce["rain_bin"] = pd.qcut(ce["rainfall"], 3, labels=["low","medium","high"], duplicates="drop")
ce["temp_bin"] = pd.qcut(ce["temperature"], 3, labels=["low","medium","high"], duplicates="drop")
ce["hum_bin"] = pd.qcut(ce["humidity"], 3, labels=["low","medium","high"], duplicates="drop")
ce["soil_bin"] = pd.qcut(ce["soil_fertility_index"], 3, labels=["low","medium","high"], duplicates="drop")
yield_q = ce["yield"].quantile([0.33,0.67]).to_numpy()
ce["risk"] = np.where(ce["yield"] <= yield_q[0], "high",
               np.where(ce["yield"] <= yield_q[1], "medium", "low"))
# Positive = high risk / low-yield seasons.
ce["positive"] = (ce["risk"] == "high").astype(int)

attrs = ["rain_bin","temp_bin","hum_bin","soil_bin"]
ce2 = ce[attrs+["positive"]].dropna().head(120).copy()
```

```
Xce = [tuple(row[a] for a in attrs) for _,row in ce2.iterrows()]
yce = ce2["positive"].to_numpy()

domains = [sorted(ce2[a].astype(str).unique().tolist()) for a in attrs]

def covers(h, x):
    return all(hv == "?" or hv == xv for hv,xv in zip(h,x))

def more_general_or_equal(h1,h2):
    return all(a == "?" or a == b for a,b in zip(h1,h2))

def min_generalize(s,x):
    out=[]
    s=list(s)
    for i,(sv,xv) in enumerate(zip(s,x)):
        if sv == "∅":
            ns=s.copy(); ns[i]=xv; out.append(tuple(ns))
        elif sv != xv:
            ns=s.copy(); ns[i]="?"; out.append(tuple(ns))
    return out

def min_specialize(g,x,domains):
    out=[]
    for i,gv in enumerate(g):
        if gv == "?":
            for val in domains[i]:
                if val != xv if False else True:
                    pass
    # explicit version for clarity
    out=[]
    for i,gv in enumerate(g):
        if gv == "?":
            for val in domains[i]:
                if val != x[i]:
                    ng=list(g); ng[i]=val; out.append(tuple(ng))
    return out

def candidate_elimination(X,y,domains):
    S={tuple("∅" for _ in domains)}
    G={tuple("?" for _ in domains)}
    for x,label in zip(X,y):
        if label == 1:
            G={g for g in G if covers(g,x)}
            newS=set()
            for s in S:
                if covers(s,x):
                    newS.add(s)
                else:
                    for gs in min_generalize(s,x):
                        if any(more_general_or_equal(g,gs) for g in G):
                            newS.add(gs)
            S=newS
            S={s for s in S if any(more_general_or_equal(g,s) for g in G)}
        else:
            S={s for s in S if not covers(s,x)}
            newG=set()
            for g in G:
                if covers(g,x):
                    for sp in min_specialize(g,x,domains):
                        if any(more_general_or_equal(sp,s) for s in S):
                            newG.add(sp)
                else:
                    newG.add(g)
            G=newG
            G={g for g in G if any(more_general_or_equal(g,s) for s in S)}
    return S,G

S_final, G_final = candidate_elimination(Xce,yce,domains)
print("Specific boundary S:")
for h in sorted(S_final): print(h)
print("General boundary G:")
for h in sorted(G_final): print(h)

# ----------------------------
# 10. SCALABILITY + APPROXIMATION (Q e)
# ----------------------------
def benchmark_knn_sizes(Xbase, ybase, sizes=(1000,5000,10000,20000)):
    rows=[]
    rng=np.random.default_rng(123)
```

```
    for n in sizes:
        if n <= len(Xbase):
            idx=rng.choice(len(Xbase), size=n, replace=False)
        else:
            idx=rng.choice(len(Xbase), size=n, replace=True)
        Xt=Xbase[idx]; yt=ybase[idx]
        q=Xt[:min(50,len(Xt))]
        t0=time.perf_counter()
        _=knn_predict(Xt,yt,q,k=5,metric="euclidean",chunk_size=64)
        elapsed=time.perf_counter()-t0
        rows.append((n,elapsed))
    return pd.DataFrame(rows,columns=["n","seconds"])

scale_df = benchmark_knn_sizes(X_train,y_train)
display(scale_df)

# Random-hyperplane LSH prototype, from scratch.
class RandomHyperplaneLSH:
    def __init__(self, dim, bits=12, seed=42):
        rng=np.random.default_rng(seed)
        self.R=rng.normal(size=(bits,dim))
        self.buckets={}

    def hash_one(self,x):
        bits=(self.R@x >= 0).astype(np.uint8)
        key=''.join(map(str,bits.tolist()))
        return key

    def fit(self,X,y):
        self.X=X; self.y=y
        self.buckets={}
        for i,x in enumerate(X):
            key=self.hash_one(x)
            self.buckets.setdefault(key,[]).append(i)

    def query(self,q,k=5):
        key=self.hash_one(q)
        cand=self.buckets.get(key,[])
        if len(cand) < k:
            return None
        cand=np.asarray(cand)
        d=np.sqrt(np.sum((self.X[cand]-q)**2,axis=1))
        kk=min(k,len(d))
        return cand[np.argpartition(d,kk-1)[:kk]]

lsh=RandomHyperplaneLSH(X_train.shape[1],bits=12)
lsh.fit(X_train,y_train)
recalls=[]
for q in X_test[:min(200,len(X_test))]:
    exact=np.argpartition(euclidean_distances(X_train,q),best_k-1)[:best_k]
    approx=lsh.query(q,best_k)
    if approx is not None:
        recalls.append(len(set(exact).intersection(set(approx)))/best_k)
print("LSH mean recall@k:", np.mean(recalls) if recalls else 0.0)

# ------------------------------
# 11. FIVE+ PUBLICATION-QUALITY VISUALIZATIONS (Q f)
# ------------------------------
plt.figure(figsize=(7,4))
plt.plot(k_values,knn_curve,marker="o")
plt.xlabel("k")
plt.ylabel("Validation RMSE")
plt.title("Manual k-NN Validation Curve")
plt.grid(alpha=.25)
plt.tight_layout()
plt.savefig("q_b_knn_validation_curve.png",dpi=220)
plt.show()

plt.figure(figsize=(7,4))
plt.bar(knn_results["Metric"],knn_results["RMSE"])
plt.ylabel("RMSE")
plt.title("Euclidean vs Mahalanobis k-NN")
plt.tight_layout()
plt.savefig("q_b_distance_comparison.png",dpi=220)
plt.show()

plt.figure(figsize=(7,4))
plt.plot(bandwidths,lwr_curve,marker="o",label="LWR")
plt.axhline(min(knn_curve),linestyle="--",label="Best k-NN RMSE")
```

```
plt.xlabel("Bandwidth h")
plt.ylabel("Validation RMSE")
plt.title("LWR Bandwidth and Bias-Variance Trade-off")
plt.legend()
plt.grid(alpha=.25)
plt.tight_layout()
plt.savefig("q_c_lwr_bias_variance.png",dpi=220)
plt.show()

plt.figure(figsize=(7,4))
plt.bar(comparison["Model"],comparison["R2"])
plt.ylabel("R²")
plt.title("k-NN vs Locally Weighted Regression")
plt.tight_layout()
plt.savefig("q_c_model_comparison.png",dpi=220)
plt.show()

# Climate-yield insights
sample = df.sample(min(4000,len(df)),random_state=42)
plt.figure(figsize=(7,4))
plt.scatter(sample["rainfall"],sample["yield"],s=8,alpha=.35)
plt.xlabel("Rainfall")
plt.ylabel("Yield")
plt.title("Rainfall vs Crop Yield")
plt.grid(alpha=.2)
plt.tight_layout()
plt.savefig("q_f_rainfall_yield.png",dpi=220)
plt.show()

plt.figure(figsize=(7,4))
plt.scatter(sample["temperature"],sample["yield"],s=8,alpha=.35)
plt.xlabel("Temperature")
plt.ylabel("Yield")
plt.title("Temperature vs Crop Yield")
plt.grid(alpha=.2)
plt.tight_layout()
plt.savefig("q_f_temperature_yield.png",dpi=220)
plt.show()

plt.figure(figsize=(7,4))
plt.plot(scale_df["n"],scale_df["seconds"],marker="o")
plt.xlabel("Training records")
plt.ylabel("Seconds")
plt.title("Brute-Force k-NN Scaling")
plt.grid(alpha=.25)
plt.tight_layout()
plt.savefig("q_e_scalability.png",dpi=220)
plt.show()

# ------------------------------
# 12. POLICY BRIEF OUTPUT
# ------------------------------
summary = {
    "records": len(df),
    "districts": df["district"].nunique(),
    "crops": df["crop"].nunique(),
    "best_k": best_k,
    "best_lwr_bandwidth": best_bw,
    "knn_rmse": float(comparison.loc[0,"RMSE"]),
    "lwr_rmse": float(comparison.loc[1,"RMSE"]),
    "lsh_recall_at_k": float(np.mean(recalls) if recalls else 0.0)
}
print(summary)
print("""
Policy brief:
1. Rainfall and temperature should be treated as risk signals, not deterministic yield controls.
2. Soil fertility indicators can help distinguish climate stress from nutrient constraints.
3. District-holdout testing is more realistic than random row splitting because it tests spatial
generalisation.
4. Lower k reduces bias but can increase variance; larger k smooths local noise.
5. LWR can model local nonlinear structure but is computationally expensive because each query solves a
weighted regression.
6. Forecasts should be accompanied by uncertainty ranges and audited for systematic errors across districts/
crops.
""")
```